Documento de Arquitectura y Especificación de Requisitos

Sistema de Control de Acceso

Ingeniería de Sistemas Integrados

Octubre 2026

Índice

1. Introducción y Topología General **2**
1.1. Topología Conceptual . . . 2

2. Análisis de Casos de Uso **3**
2.1. Definición de Actores . . . 3
2.2. Especificación Detallada de Casos de Uso . . . 3

3. Especificación de Requisitos y Trazabilidad **4**
3.1. Requisitos Funcionales (RF) . . . 4

4. Diseño Arquitectónico del Pipeline (GStreamer) **4**
4.1. Justificación de Restricciones (Capsfilters) . . . 5
4.2. Presupuesto de Latencia Teórica . . . 5

5. Lógica arquitectura de Software (Python) **6**
5.1. Orquestación y Pipeline (`main.py` y `pipeline_gstreamer.py`) . . . 6
5.2. Hardware y Estados (`gpio_ctrl.py` y `estado_puerta.py`) . . . 6
5.3. Inteligencia Artificial y Eventos (`hilo_ia.py`) . . . 6
5.4. Telemetría y Auditoría (`bitacora.py` y `salud.py`) . . . 7

6. Memoria de Ingeniería: Cronograma de Depuración y *Hot-Patching* **7**
6.1. Hito 1: Aislamiento de Red y Bloqueo PAM (SSH) . . . 7
6.2. Hito 2: Evasión del Cortafuegos y Aislamiento IP . . . 7
6.3. Hito 3: Pánico del Codificador y Riesgo Térmico . . . 7
6.4. Hito 4: Lógica Invertida del Actuador REX . . . 8
6.5. Hito 5: Fuga de Evidencia Forense (Memoria Volátil) . . . 8
6.6. Conclusión de Operaciones . . . 8

1. Introducción y Topología General

El presente documento detalla la arquitectura, la especificación de requisitos y el diseño de un Sistema de Control de Acceso Físico Inteligente. Este sistema ha sido construido sobre hardware de grado embebido (SoC BCM2711 en una Raspberry Pi 4) operando bajo un sistema operativo Linux minimalista compilado a la medida mediante **Yocto Project** (rama *Scarthgap/Wrynose*). El presente documento detalla la arquitectura, la especificación de requisitos y el diseño de un Sistema de Control de Acceso Físico Inteligente. Este sistema ha sido construido sobre hardware de grado embebido (SoC BCM2711 en una Raspberry Pi 4) operando bajo un sistema operativo Linux minimalista compilado a la medida mediante **Yocto Project** (rama *Wrynose 6.0*)

1.1. Topología Conceptual

La infraestructura se divide arquitectónicamente en tres dominios físicos y lógicos, diseñados para garantizar la degradación segura y la tolerancia a fallos:

1. **Capa de Hardware en el Borde:** Compuesta por la Raspberry Pi 4, la cual interactúa con el mundo físico mediante captura de video UVC y el cabezal GPIO. El hardware controla un relé de apertura (LED Verde), un indicador de estado de bloqueo (LED Rojo) y un lazo de seguridad para evacuación manual (Botón REX).
2. **Capa de Red Perimetral:** El sistema se acopla a la red corporativa mediante `systemd-networkd` (DHCP). Utiliza el protocolo asíncrono UDP para el transporte de paquetes RTP que contienen el video comprimido en el estándar H.264.
3. **Capa de Monitoreo Remoto:** Una estación de trabajo externa que recibe el flujo de video en vivo. Esta estación está totalmente desacoplada lógicamente de la toma de decisiones.

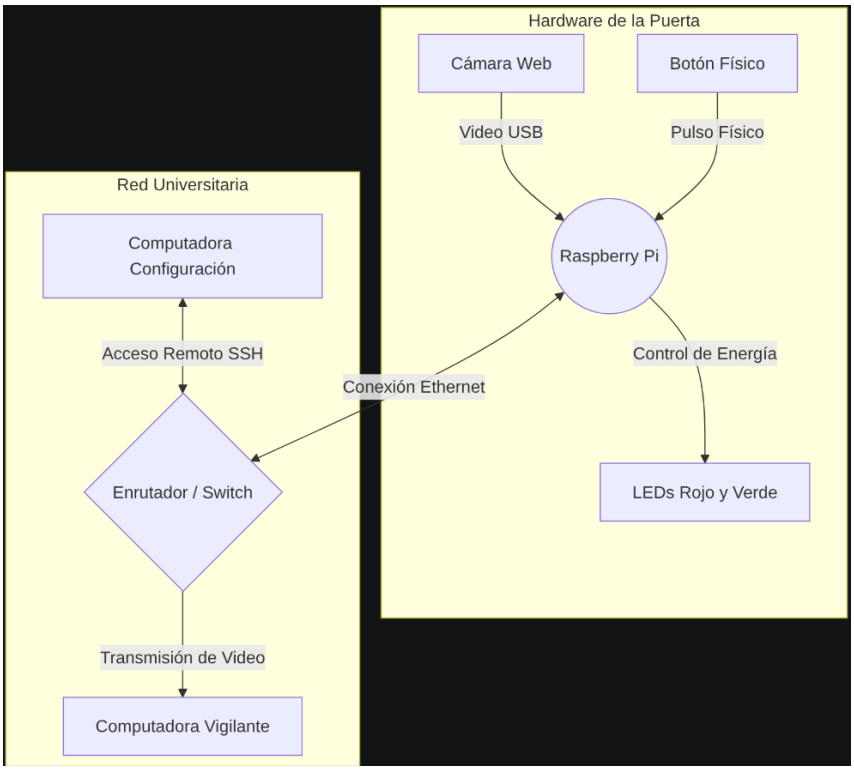


Figura 1: Diagrama General de Topología Física y Red

2. Análisis de Casos de Uso

2.1. Definición de Actores

- **Ingeniero:** Personal técnico responsable del despliegue del firmware. Configura variables volátiles en el archivo de entorno.
- **Vigilante:** Operador de la estación de seguridad central. Recibe de manera pasiva el video en tiempo real.
- **Persona 1 (Autorizada):** Usuario legítimo que requiere ingresar o salir.
- **Persona 2 (Intruso):** Entidad no autorizada que intenta vulnerar el perímetro ingresando furtivamente.

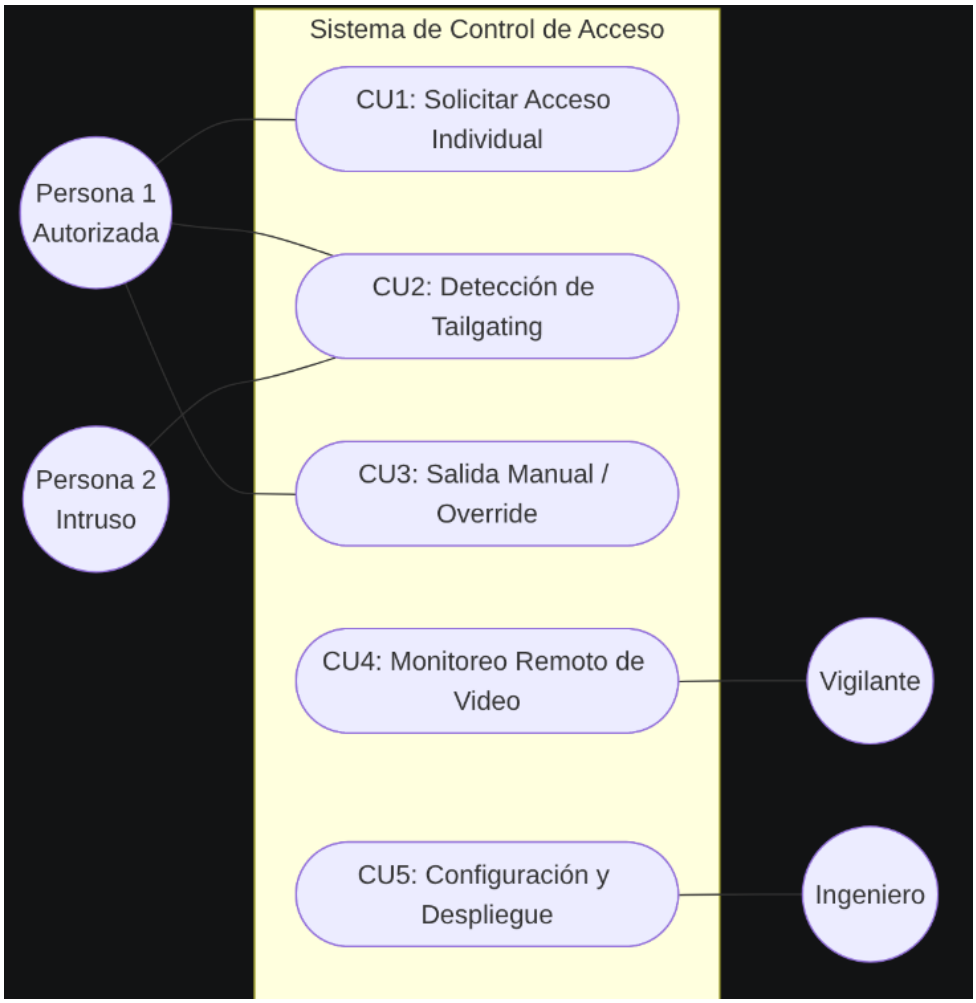


Figura 2: Diagrama de Casos de Uso del Sistema

2.2. Especificación Detallada de Casos de Uso

- **CU1 (Acceso Individual):** El sistema aísla un (1) solo rostro en el cuadro, autoriza el acceso activando el relevador por 3.0 segundos y asienta el evento en la bitácora.
- **CU2 (Detección de Tailgating):** La IA devuelve un array con más de un rostro. Se bloquea la apertura, se dispara un evento para capturar evidencia fotográfica en el disco duro, y se levanta una alerta en el registro CSV.

- **CU3 (Salida Manual / Override):** La Persona 1 acciona el botón físico REX. El sistema detecta una interrupción de hardware (`FALLING_EDGE`) en el GPIO, liberando la cerradura eléctrica por 3.0 segundos. *Nota de Prototipo: Debido a limitaciones de componentes, se implementó con un pulsador Normalmente Abierto (NA), careciendo de seguridad Fail-Safe ante cortes de cableado.*
- **CU4 (Monitoreo Remoto):** La Raspberry Pi codifica por hardware a H.264 y envía los paquetes UDP. Todo esto ocurre en un hilo separado de la IA.
- **CU5 (Despliegue Zero-Touch):** `systemd` inicializa los servicios, espera 5 segundos a que la cámara USB se registre, e inicia la aplicación en modo autónomo (`Restart=always`).

3. Especificación de Requisitos y Trazabilidad

3.1. Requisitos Funcionales (RF)

ID	Descripción del Requisito	Trazabilidad (Módulo)
RF-01	**Acceso Individual:** La decisión de acceso debe correr fuera del hilo principal de GStreamer para evitar bloqueos.	`hilo_ia.py` y `appsink pull-sample`
RF-02	**Tiempo de Decisión:** El sistema negará el acceso por omisión si pasan 10s sin detección.	`hilo_ia.py` (`TIMEOUT`)
RF-03	**Prevención de Tailgating:** Detección de 2 o más rostros simultáneos denegará el acceso.	`hilo_ia.py` (`len(cajas) >1`)
RF-04	**Botón Override:** Existirá un botón de salida manual configurado con lógica tolerante a fallos y temporizador 3.0s.	`gpio_ctrl.py` (`FALLING_EDGE`)
RF-05	**Evidencia Forense:** Registros y capturas fotográficas ante violaciones deben persistir en almacenamiento físico.	`bitacora.py` y `Pad Probe`
RF-06	**Manejo de EOS:** Al detener el servicio se enviará señal EOS para asegurar la integridad de la última fotografía.	`main.py` (`_apagado_ordenado`)

4. Diseño Arquitectónico del Pipeline (GStreamer)

El núcleo multimedia fue diseñado bajo el principio de asincronía estricta. Utilizando GStreamer, la topología extrae el video de la fuente y lo bifurca hacia tres consumidores independientes con capacidades y formatos diametralmente distintos.

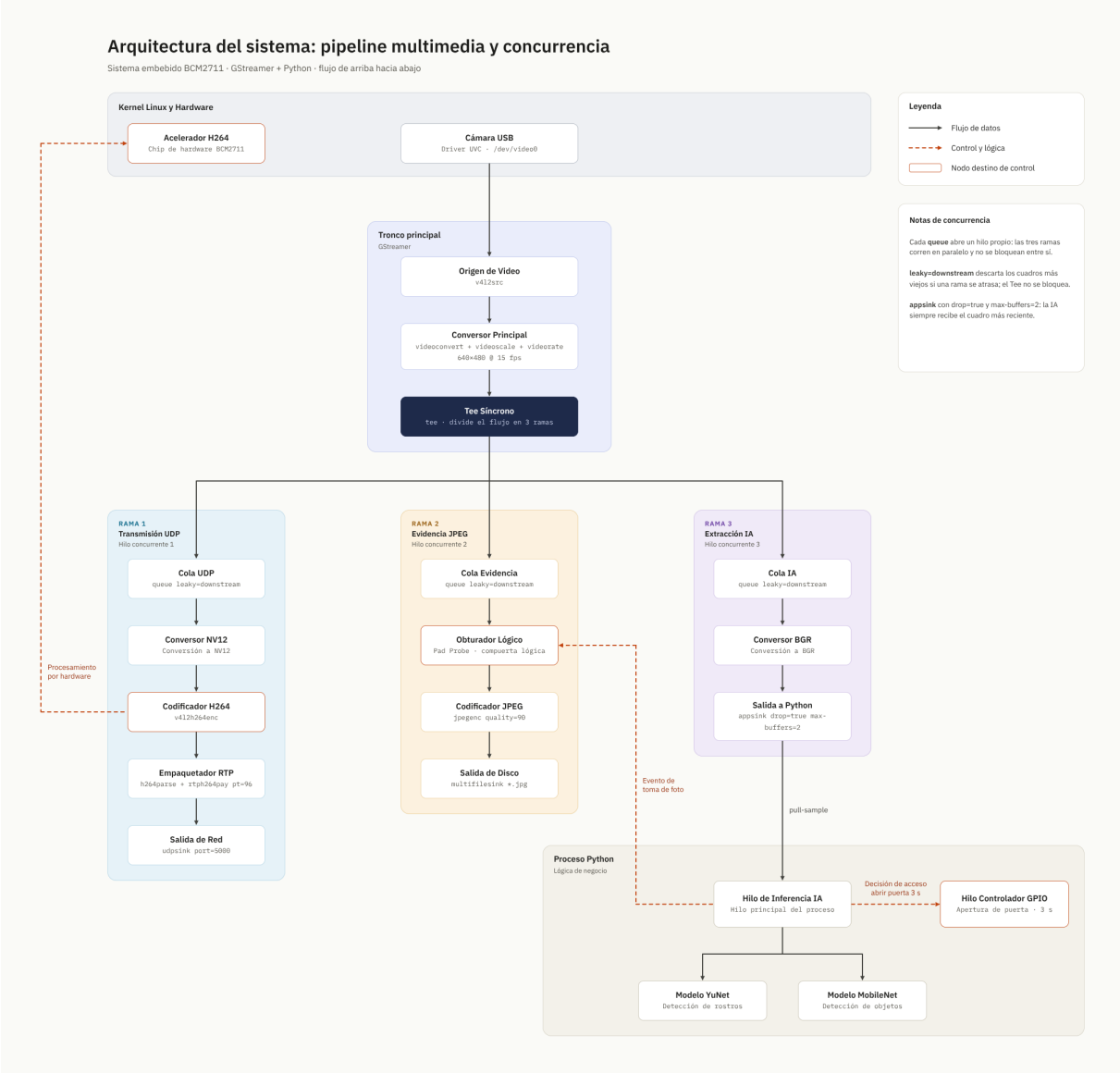


Figura 3: Diagrama de Flujo Interno y Concurrencia del Pipeline Multimedia

4.1. Justificación de Restricciones (Capsfilters)

- `video/x-raw, width=640, height=480, framerate=15/1`: Se limitó la fuente original a resolución VGA a 15 fps. Procesar 1080p a 30fps mediante IA por CPU saturaría los núcleos ARM, induciendo temperaturas críticas.
- `video/x-raw, format=NV12`: Restricción inyectada antes del codificador. El bloque físico V4L2 M2M del chip de Raspberry Pi solo acepta memoria en formato NV12.

4.2. Presupuesto de Latencia Teórica

El diseño de concurrencia (`queue leaky=downstream`) garantiza una latencia End-to-End inferior a 250 ms.

- **Fuente (UVC):** ~66 ms (1 cuadro a 15 fps).
- **Buffers (Queues asíncronos):** ~66 ms.

- **Codificador (Hardware `v4l2h264enc`):** ∼30 ms.
- **Transporte (Red UDP Router TEC):** ∼15 ms.
- **Decodificador (Windows Direct3D11):** `avdec_h264` y renderizado Direct3D11 ∼30 ms.
- **Latencia Total Estimada: ∼207 ms**.

5. Lógica arquitectura de Software (Python)

El código de aplicación actúa como el cerebro supervisor del sistema. La arquitectura fue diseñada siguiendo el principio de Responsabilidad Única (SOLID), dividiendo la lógica en 10 módulos especializados. No procesa video directamente en el hilo principal, sino que orquesta los *callbacks* de GStreamer y los actuadores físicos bajo el estándar de la biblioteca `libgpiod` v2.

5.1. Orquestación y Pipeline (`main.py` y `pipeline_gstreamer.py`)

El módulo `main.py` inicializa el `GLib.MainLoop` y conecta las señales del sistema operativo para garantizar un apagado ordenado enviando una señal `EOS` (End of Stream). Esto asegura que los buffers de video en memoria se vacíen al disco duro antes de apagar, evitando archivos corruptos. Por su parte, `pipeline_gstreamer.py` extrae los *frames* mediante un `appsink` y los envía a una cola `Queue` *Thread-Safe* de Python, desacoplando completamente los hilos en C de GStreamer de los hilos de inferencia en Python.

5.2. Hardware y Estados (`gpio_ctrl.py` y `estado_puerta.py`)

Para manejar las señales físicas, el sistema reserva los pines GPIO a través del `gpiochip0`.

- **Gestión de Estados:** `estado_puerta.py` utiliza bloqueos de exclusión mutua (`threading.RLock`) para evitar condiciones de carrera cuando la IA y el botón físico intentan modificar el estado del relé simultáneamente.
- **Interrupciones Físicas:** El Botón de Salida (REX) se procesa en un hilo independiente (`hilo_override`). Al tratarse de un pulsador momentáneo, se configuró para detectar el `FALLING_EDGE`, eliminando la necesidad de ciclos de lectura continua (*polling*) y liberando CPU.

5.3. Inteligencia Artificial y Eventos (`hilo_ia.py`)

El motor de decisión extrae fotogramas en formato BGR y ejecuta dos redes de forma paralela:

1. **YuNet ONNX:** Optimizado para detección facial de alta velocidad. Emite conteos de rostros. Su regla de negocio estipula que detectar $N = 1$ rostro autoriza la apertura (3.0s), mientras que detectar $N > 1$ rostros levanta la excepción de *Tailgating*, activando el *Pad Probe* para capturar evidencia fotográfica.
2. **MobileNet-SSD (Caffe):** Funciona como clasificador de contexto (20 clases). Infiere la presencia de objetos (ej. animales, vehículos) registrándolos en la bitácora para analítica, sin interferir con las decisiones críticas de la puerta.

5.4. Telemetría y Auditoría (`bitacora.py` y `salud.py`)

- **Persistencia Legal:** `bitacora.py` registra cada evento en un archivo CSV. Tras cada escritura, invoca `os.fsync()` para forzar físicamente el volcado de la memoria caché al disco, asegurando que la evidencia no se pierda ante cortes eléctricos.
- **Watchdog:** `salud.py` reporta métricas cada 5 segundos. Audita el consumo de memoria y envía un latido al demonio `systemd` (`sd_notify`); si el hilo de IA se congela, Yocto reinicia el sistema autónomamente.

6. Memoria de Ingeniería: Cronograma de Depuración y *Hot-Patching*

El despliegue de un sistema *bare-metal* en un entorno corporativo hostil conllevó múltiples condiciones de falla. A continuación, el reporte *Post-Mortem* de la puesta en marcha.

6.1. Hito 1: Aislamiento de Red y Bloqueo PAM (SSH)

- **Fallo:** La imagen Yocto no solicitaba IP vía DHCP y el módulo de seguridad PAM rechazaba la conexión SSH.
- **Análisis de Causa:** Yocto compila sin gestor de red activo por omisión. Además, el módulo Pluggable Authentication Modules (PAM) bloquea cuentas sin contraseña, ignorando directivas permisivas en `sshd_config`.
- **Resolución Final:** Se inyectó una Llave Criptográfica Pública (Ed25519) en el `rootfs` para saltar el control PAM, y se forzó la habilitación de `systemd-networkd` mediante *symlinks*.

6.2. Hito 2: Evasión del Cortafuegos y Aislamiento IP

- **Fallo:** El firewall de la universidad bloqueaba las peticiones HTTP de salida (Webhooks), haciendo imposible descubrir la IP dinámica asignada a la placa.
- **Análisis de Causa:** Los puertos del edificio asignaban a la placa una subred distinta, haciéndola invisible a escaneos locales.
- **Resolución Final (Técnica *Sneakernet*):** Se programó un *script* temporal de arranque que consultaba la interfaz `eth0` y escribía la IP en la partición FAT32 (`/boot/ip_secreta.txt`). Al leer la SD externamente, se reveló la IP real.

6.3. Hito 3: Pánico del Codificador y Riesgo Térmico

- **Fallo:** GStreamer abortó con error `not-negotiated`. El pipeline requería Buffers DMA exclusivos de la cámara oficial Raspberry Pi. Al usar cámara USB, colapsó.
- **Resolución Final:** Se rediseñó el pipeline inyectando conversores de video explícitos. Esto permitió rehabilitar el Acelerador de Hardware (`v4l2h264enc`), reduciendo el consumo de CPU a un 2.0 % y estabilizando la temperatura de la placa en 71.0 °C.

6.4. Hito 4: Lógica Invertida del Actuador REX

- **Fallo:** El botón físico de salida (REX) no accionaba los relés lógicos.
- **Análisis de Causa:** Se conectó un pulsador Normalmente Abierto (NA) cuando la arquitectura requería Normalmente Cerrado (NC).
- **Resolución Final:** Se mapeó el controlador al chip correcto en Linux 6.x (`gpiochip0`) y se invirtió la polaridad de interrupción a `FALLING_EDGE` en el archivo `gpio_ctrl.py`, enlazándolo al relé temporizado.

6.5. Hito 5: Fuga de Evidencia Forense (Memoria Volátil)

- **Fallo:** Las fotografías capturadas de Tailgating desaparecían al reiniciar la placa. Yocto monta ciertas rutas del sistema como un disco RAM (`tmpfs`).
- **Resolución Definitiva:** Se migró la variable estructural `ACCESO_DATA` global hacia la ruta `/home/root/`, garantizando persistencia ext4 física a prueba de apagones.

6.6. Conclusión de Operaciones

Las cinco crisis expuestas fueron solventadas en terreno mediante intervenciones *Hot-Patching*. En la fase final, todas las correcciones fueron abstraídas e inyectadas retrospectivamente en la receta de Yocto (`control-acceso_1.0.bb`, `local.conf` y archivo `.env`), consolidando la arquitectura final en un formato *Single Source of Truth* capaz de operar en frío (*Zero-Touch*) ante cualquier despliegue futuro.